

De l'idée au code : et si l'IA était là *à chaque étape* ?

Méthodologie BMAD : Une méthode pour reprendre la main sur ce que l'IA produit, du brief au merge.



Yohann Ciurlik

Principal Solutions Architect · DCX Paris



Giovanni Scamandro

Managing Solutions Architect · DCX Lille



SPEAKERS

Qui sommes-nous ?

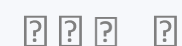
Yohann Ciurlik

PRINCIPAL SOLUTIONS ARCHITECT

Capgemini

Digital Customer Experience Paris

Architecte solutions, passionné de DIY (domotique, impression 3D, ...) et de projets open sources.



Giovanni Scamandro

MANAGING SOLUTIONS ARCHITECT

Capgemini

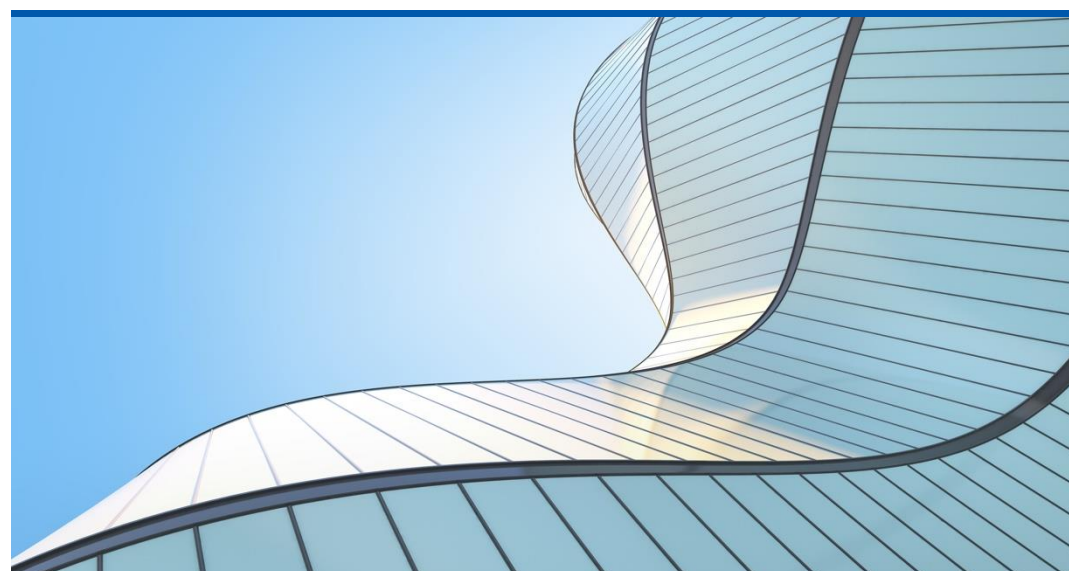
Digital Customer Experience Lille

Architecte solutions, j'accompagne les projets de l'idée à la réalisation, tout en faisant monter les équipes en compétences.



CONTEXTE

Entité d'expertise technique



Notre structure

ORGANISATION

50 collaborateurs

Répartis en pôles d'expertise techniques et communautés d'innovation dans 8 villes : Paris, Rennes, Toulouse, Brest, Lyon, Nantes, Aix-en-Provence et Lille.



Nos valeurs

ADN

Entraide · Curiosité · Communication

Nous valorisons compétences techniques et interpersonnelles. Polyvalence et soft skills sont au cœur de notre ADN.



Nos enjeux

MISSION

Accompagnement · Stratégie · Mise en œuvre

Accompagnement de bout en bout, de la définition de la stratégie technologique à la mise en œuvre des solutions, jusqu'à la formation des équipes.



34 MINUTES

Plan du talk

01

Le problème

Pourquoi structurer ses échanges avec l'IA ?

5 min

02

BMAD

Ce que c'est, comment ça marche

7 min

03

Les fonctionnalités clés

Ce qui fait vraiment la différence au quotidien

15 min

04

Démo — HoleSocks

La boutique de chaussettes déjà trouées

7 min



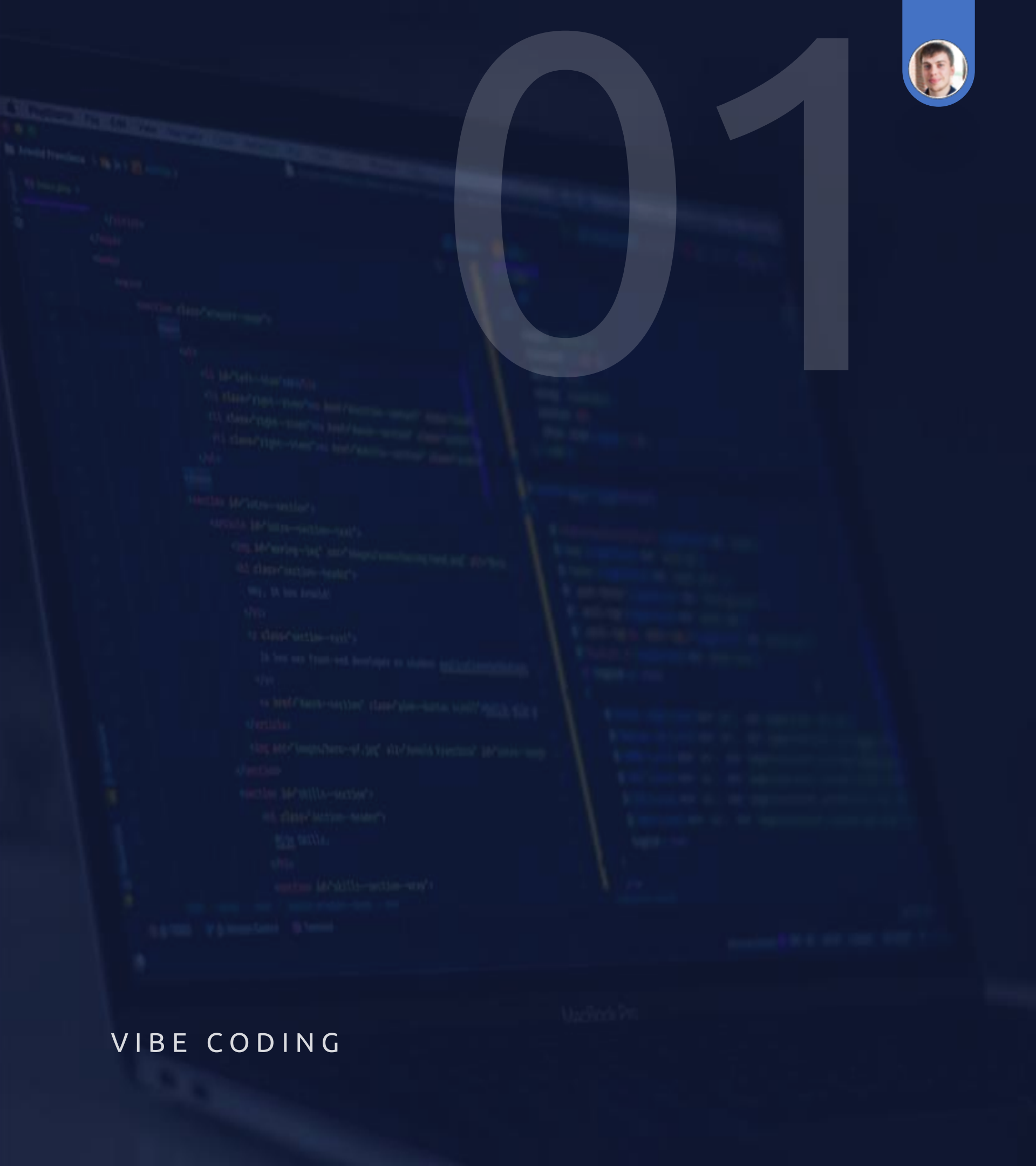
01

01 / 04

SECTION

Le problème.

Pourquoi structurer ses échanges avec l'IA ?



VIBE CODING



LE PROBLÈME AVEC LE VIBE CODING

L'IA génère du code. Mais lequel ?

VIBE CODING

« *On prompt, on croise les doigts, et on recommence.* »

01

Pas de mémoire

L'IA repart de zéro à chaque conversation. Vos décisions d'hier sont perdues.

02

Résultats aléatoires

La même demande donne des résultats différents — impossible de capitaliser.

03

Coût caché

Les erreurs se découvrent tard.
Corriger en sprint 3 coûte 10× plus.

04

Perte de contrôle

L'IA part dans une direction, vous dans une autre — personne ne pilote.



LA RÉPONSE : LE SPEC-DRIVEN DEVELOPMENT

Écrire la spec avant le code.

Une spécification partagée entre vous et l'IA. Fil conducteur, mémoire commune, référence pour tout ce qui suit.

LES 3 NIVEAUX SELON MARTIN FOWLER

NIVEAU 1

Spec-first

La spec guide

Un document écrit avant de coder. L'IA le lit et s'en inspire.



NIVEAU 2

Spec-anchored

La spec pilote

Document maintenu tout au long du projet. L'IA ne code pas sans elle.



NIVEAU 3

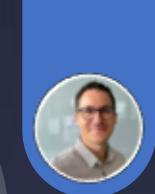
Spec-as-source

La spec est le code

L'humain n'écrit plus que la spec. Le code en est la compilation.

BMAD vise le niveau 2 → 3 : spec vivante, agents spécialisés, trace complète de chaque décision.

Martin Fowler est Chief Scientist chez ThoughtWorks, auteur de *Refactoring* et de *Patterns of Enterprise Application Architecture*.
Figure mondiale de l'ingénierie logicielle, il a popularisé l'idée que la qualité de la spec est le premier levier de qualité du code.



02

02 / 04

SECTION

BMAD.

Ce que c'est, comment ça marche concrètement.

BUILD MORE ARCHITECT DREAMS



DÉFINITION

BMAD — Build More Architect Dreams.

Une équipe projet complète, en agents IA.

De l'idée au code : chaque agent a un rôle précis et se passe le travail selon un workflow structuré.

43K+

étoiles GitHub

Open source

Gratuit

Compatible

Claude, Cursor, Copilot...

Ce que c'est

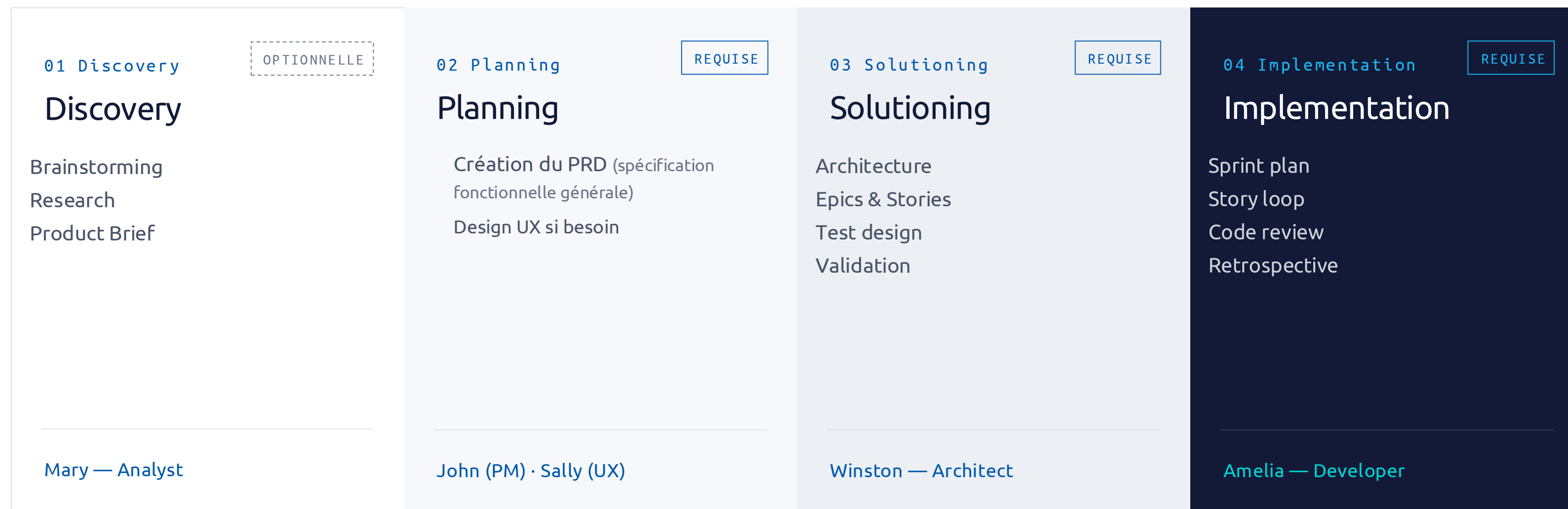
- ✓ Un framework de pilotage de projet avec l'IA
- ✓ Une méthode pour aller de l'idée au code sans perdre le fil
- ✓ Un système qui garde la mémoire de toutes vos décisions
- ✓ Compatible Claude, Cursor, VS Code, Copilot...
- ✓ Extensible et customisable

Ce que ce n'est pas

- ✗ Un assistant IA classique à qui on pose des questions
- ✗ Un outil réservé aux développeurs seniors
- ✗ Une solution pour remplacer l'équipe
- ✗ Obligatoire pour chaque projet — il y a des alternatives

WORKFLOW

Les 4 phases — de l'idée au code.



La phase 4 tourne en boucle : **Create Story** › **Develop** › **Code Review** — jusqu'à la fin du projet.

L'ÉQUIPE

Chacun son rôle.



M

Mary

ANALYST

Brainstorm / Research
Product Brief / PRFAQ



J

John

PRODUCT MANAGER

PRD / Epics / Stories
Validation des exigences



W

Winston

ARCHITECT

Architecture technique
Choix et trade-offs



A

Amelia

DEVELOPER

Dev Stories / Code Review
Quick Dev / Tests



S

Sally

UX DESIGNER

Design d'interface
Parcours utilisateur



P

Paige

TECH WRITER

Documentation
Guides et standards



BMad Master Orchestrator — coordonne les passages entre agents et garantit la cohérence des livrables.



03

03 / 04

SECTION

Les fonctionnalités clés.

Ce qui rend BMAD vraiment utile au quotidien.

DISCOVERY · PARTY MODE · ADVERSARIAL ·
QUICK DEV · CHECKPOINT

**DISCOVERY**

Clarifier avant de construire.

De manière itérative, l'agent pose des questions sur le contexte, la vision, les personas — pour préciser le besoin avant toute solution technique.

Ce que ça change : le besoin est compris et validé **avant** de parler technique. Moins de revirements de cap, plus de valeur.

Cadrage métier

L'agent questionne la valeur, les utilisateurs, les contraintes — comme un consultant qui onboarde.

Vision produit

Challenger les hypothèses : pourquoi cette feature ? Pour qui ? Qu'est-ce qui change si on ne la fait pas ?

Personas & contexte

Identifier les vrais usages, les douleurs, les attentes — sans partir de solutions préconçues.

Exploration d'idée

Transformer un concept flou en besoin structuré, prêt à devenir un PRD solide.



DISCOVERY

Clarifier une idée floue.

Situation : vous arrivez avec une idée — « *je veux faire une appli pour les sportifs* » . Mary entre en scène.

Mary

ANALYST

Quels sportifs précisément ? Amateur du dimanche, athlète en préparation de compétition, ou coach qui suit plusieurs personnes ?

Vous

PRODUCT OWNER

Plutôt des amateurs, qui courent 2-3 fois par semaine et veulent progresser.

Mary

ANALYST

Quel est leur vrai problème aujourd'hui ? Manque de motivation ? Mauvaise progression ? Pas de plan adapté ?

Vous

PRODUCT OWNER

Ils ne savent pas s'ils progressent vraiment. Ils courent mais sans repères.

Mary

ANALYST

Donc le besoin core c'est : **visualiser sa progression**. Pas une appli sportive générique — un outil de feedback personnel.

Résultat : en 5 questions, l'idée floue devient un besoin précis. Le PRD sera 10× plus pertinent.

**PARTY MODE**

Décider mieux, sans réunion de 2h.

Vous posez une question complexe. Vos agents — PM, Architecte, Dev, UX — la débattent en temps réel, dans une seule conversation.

Ce que ça change : des décisions architecturales en 5 min au lieu de 2 jours de réunion.

01

Lancer la commande

Vous tapez la commande dédiée et formulez votre question.

```
/bmad-party-mode
```

02

BMad Master orchestre

Choisit les agents pertinents pour la question posée.

03

Débat en temps réel

Les agents s'accordent ou s'affrontent — chacun avec son angle.

04

Vous guidez

Questions, relances — vous gardez la main et vous tranchez.

Utile pour : choix techniques · post-mortems · planification de sprint · brainstorming.



PARTY MODE

Post-mortem de panne.

Vous
TECH LEAD

Le service authentication est tombé 3 fois ce sprint. Qu'est-ce qui s'est passé ?

Winston
ARCHITECTE

Le design était solide. Si l'implémentation a dévié du plan technique...

Amelia
DEVELOPER

J'ai suivi le plan. Mais le plan ne couvrait pas les conflits d'accès simultanés sur les sessions.

John
PRODUCT MANAGER

La vraie cause : les exigences sur la gestion de session n'étaient pas dans le document initial.

Résultat : chaque partie prenante a identifié sa part. En 5 minutes, sans réunion.

ADVANCED ELICITATION

Trouver les trous avant de coder.

L'IA vient de générer un document. Vous choisissez une méthode pour qu'elle relise son propre travail avec un angle critique.

Ce que ça change : les problèmes remontent **avant** de coder. Pas 3 sprints plus tard.

QUELQUES MÉTHODES DISPONIBLES

Pre-mortem

TOP

Imaginez que le projet a échoué. Pourquoi ? Listez les causes plausibles avant qu'elles arrivent.

Inversion

Comment garantir l'échec ? Une fois la liste faite — évitez exactement ça.

Red Team

TOP

Attaquez votre propre plan, puis défendez-le. Deux postures, le même document.

Premiers principes

Tout supprimer, repartir de zéro. Si on devait redessiner aujourd'hui — qu'est-ce qu'on garderait ?

Parties prenantes

Réévaluer depuis le point de vue de chaque acteur — métier, ops, sécurité, utilisateur final.

Sans contraintes

Sans aucune limite — budget, temps, équipe — que changerait-on ? Ce qui sort, c'est l'idéal.

ADVERSARIAL REVIEW

Votre relecteur le plus impitoyable.

LA RÈGLE

« Tu dois trouver des problèmes. Aucun problème trouvé = refus de valider. Recommence. »

L'IA adopte une posture critique — elle assume que des problèmes existent et cherche jusqu'à les trouver.

Ce que ça change : vous mergez du code sans faille de sécurité évidente. Pas après la prod.

Brise l'approbation automatique

« Ça me semble correct » ne suffit plus — l'IA doit prouver qu'elle a cherché.

Cherche ce qui manque

Pas seulement les erreurs — les oublis aussi. Les exigences non couvertes.

Résultats concrets

Chaque problème trouvé est localisé, contextualisé, expliqué.

Regard neuf

Relecture sans accès au raisonnement d'origine — comme un nouveau relecteur.

ADVERSARIAL REVIEW

Module de connexion.

Contexte : Amelia vient de développer le module de connexion. Adversarial Review activé.

RELECTURE CLASSIQUE

« *Le module de connexion semble correct.* »

✓ **APPROUVÉ**

ADVERSARIAL REVIEW

HAUT

Pas de limite sur les tentatives — brute force possible.

HAUT

Jeton stocké dans le navigateur → exposé aux scripts (XSS).

HAUT

Vérification du mot de passe uniquement côté client.

MOY

Aucune trace des échecs de connexion — pas d'audit possible.

BAS

Valeur 3600 en dur — devrait être une constante nommée.

4 failles détectées avant la mise en ligne.



QUICK DEV · CHECKPOINT

Quick Dev & Checkpoint Preview.

Quick Dev

Zéro lourdeur pour les petits changements. Corriger un bug sans passer par tout le workflow.

- Correction de bug isolée
- Petite évolution sans impact sur l'archi
- Changement de config ou d'interface
- Prototype rapide à tester

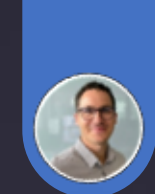
[bmad-agent-dev](#) › QD › spec 1 page › code direct

Checkpoint Preview

Valider chaque transition de phase avant de continuer. Évite de construire sur une base fragile.

- ✓ Les documents sont cohérents entre eux
- ✓ Rien d'important n'a été oublié
- ✓ Pas de contradiction plan / archi
- ✓ L'équipe est prête à coder

Évite de découvrir en sprint 3 ce qui aurait dû être décidé en sprint 1.



04

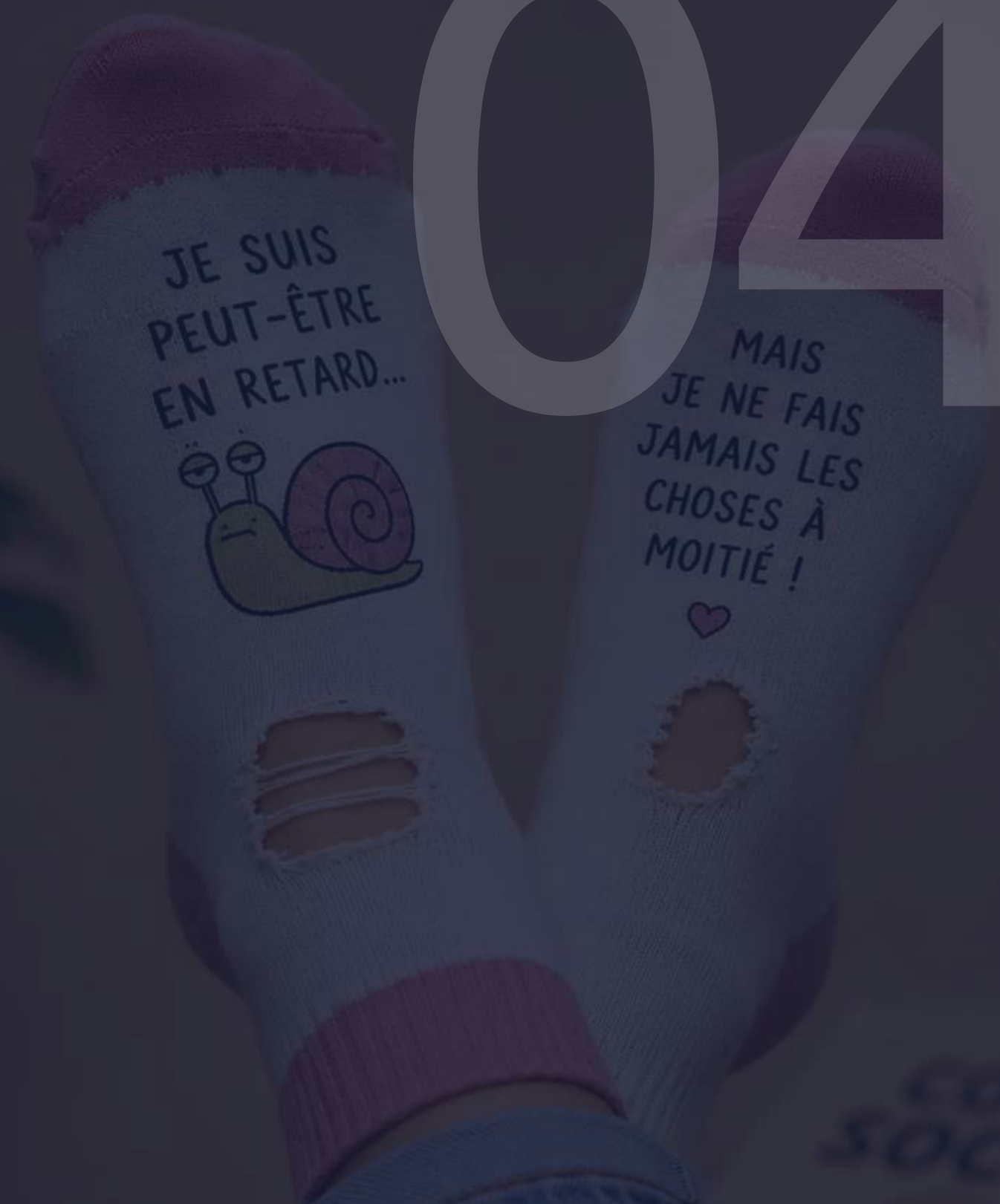
04 / 04

SECTION

Démo — HoleSocks.

La boutique de chaussettes déjà trouées.

DE L'IDÉE AU CODE, EN LIVE



HOLESOCKS

L'idée la plus absurde possible, exprès.

« *Pourquoi attendre l'usure ?* »

Une boutique en ligne de chaussettes vendues déjà trouées.



CATALOGUE

3 niveaux de trou : Léger, Aéré, Catastrophe

FILTRES

Par taille de trou, ventilation, couleur

AVIS CLIENTS

« Mon pied respire enfin » — 5 étoiles

UPSELL

Kit pour agrandir les trous soi-même



Discovery & Planning.

PHASE 1 · DISCOVERY

Brainstorm avec Mary

› Mary : « Aide-moi à affiner l'idée HoleSocks »

Insight clé : le catalogue doit avoir un « niveau de trou » — c'est le différenciateur.
Fonctionnalité non évidente : l'upsell « kit à trouer soi-même ».

PHASE 1 · DISCOVERY

Product Brief

› Mary : « Génère le Product Brief »

Vision, cibles, valeur unique, périmètre — en 2 pages. Décision : on se limite à la vente en ligne, pas de B2B.

PHASE 2 · PLANNING

PRD avec John

› John : « Crée le PRD à partir du Brief »

Exigences fonctionnelles, critères d'acceptation. Advanced Elicitation (pre-mortem) → 3 gaps trouvés avant de continuer.

PHASE 2 · PLANNING

Party Mode — débat technique

› Toute l'équipe : « Besoin d'un moteur de reco pour les trous ? »

Winston : trop tôt, MVP d'abord. John : inutile si catalogue < 50 produits. Amelia : +3 semaines de dev pour 0 valeur. Décision : non.



Solutioning & Implementation.

PHASE 3 · SOLUTIONING

Architecture Winston

› Winston : « Crée l'architecture pour HoleSocks »

Stack choisi, découpage en services, modèle de données. Adversarial Review → failles trouvées : gestion stocks non traitée, paiement non sécurisé.

PHASE 3 · SOLUTIONING

Epics & Stories

› John + Winston : « Génère les epics et les user stories »

12 stories priorisées avec critères d'acceptation et contexte technique embarqué. Chaque story est autonome — Amelia peut coder sans question.

PHASE 4 · IMPLEMENTATION

Développement Amelia

› Amelia : « Dev Story #1 — Catalogue produits avec niveau de trou »

Code généré, tests inclus, documentation inline. Code Review (Adversarial) → 2 remarques mineures corrigées avant merge.

PHASE 4 · RÉSULTAT

Résultat final

› En combien de prompts a-t-on une base de projet complète ?

Brief / PRD / Architecture / 12 stories / Premier composant codé et testé. Sans BMAD : idée floue → code → refactor → idée encore floue.

HOLESOCKS · PHASES 1 & 2

Améliore l'UX/UI

Sélectionnez vos chaussettes préférées

Explorez notre collection de chaussettes trouées pour tous les niveaux

Affinez votre recherche

NIVEAU DE TROU

Tous les niveaux

Léger

Aéré

Catastrophe

COULEUR

Toutes les couleurs

Rouge

Bleu

Noir

Blanc

STYLE DE TROU

Tous les styles

Orteil

Talon

Semelle

TAILLE

Toutes les tailles

S

M

L

XL

Réinitialiser les filtres

8 produits

Le Philosophe

Trou parfait pour la méditation 🧘

Niveau: ⭐

Voir les détails →

Le Drôle

Fait rire les pieds... du moins ce qu'il en reste 😂

Niveau: ⭐⭐

Voir les détails →

Le Coquin

Presque plus de chaussette, c'est l'idée 🤪

Niveau: ⭐⭐⭐

Voir les détails →

Le Bricoleur

Un trou pour tous vos besoins de bricolage 🛠

Niveau: ⭐⭐

Voir les détails →

Le Minimal

Un trou, c'est assez pour tout 🌈

Niveau: ⭐

Le Extrême

Plus de trous que de tissu 🌀

Niveau: ⭐⭐⭐⭐

Options pour améliorer l'UI

Option 1 — Recommandée si tu veux du résultat solide et structuré

[CU] Create UX — bmad-create-ux-design

→ Te guide pour définir les patterns visuels, la hiérarchie, les composants, l'identité graphique du site. Produit un ux-design.md qui servira de référence aux agents de dev.

⚠ Ce step a été sauté. Partir de là te donnera une base claire pour toutes les itérations UI suivantes.

Option 2 — Recommandée si tu veux des changements rapides maintenant

[QQ] Quick Dev — bmad-quick-dev

→ Tu décris ce que tu veux améliorer (couleurs, animations, typographie, layout...) et l'agent implémente directement dans le code. Pas de doc, juste du code.

Idéal si tu as une vision précise de ce que tu veux changer et tu veux avancer vite.

Option 3 — Si tu veux reformuler formellement

[CC] Correct Course — bmad-correct-course

→ Pour intégrer un redesign UI comme un changement officiel dans le sprint en cours, avec mise à jour des epics/stories.

Ma recommandation

Tu n'as jamais fait l'étape UX — je te propose de lancer bmad-create-ux-design maintenant. L'agent Sally va te guider pour définir l'identité visuelle du site HoleSocks (palette, typographie, composants, animations), puis on implémente.

Veux-tu que je lance [CU] Create UX pour toi ?



À RETENIR

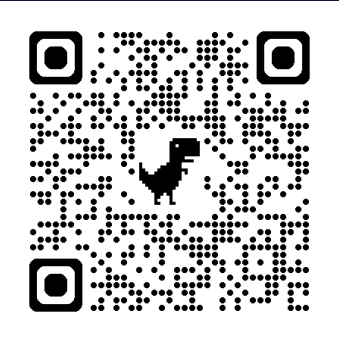
Ce que cela apporte

- 01 Un **fil conducteur** de l'idée au code — sans jamais perdre le contexte.
- 02 Des **décisions architecturales en quelques minutes**, pas en jours.
- 03 Des **failles détectées avant de coder**, pas après la mise en prod.
- 04 Une **documentation vivante** générée au fil du projet.
- 05 **Moins de réunions**. Chaque agent joue son rôle, vous gardez la main.

Mais attention à ...

- 01 La **courbe d'apprentissage** et le **paradoxe d'accessibilité** (tech vs non-techs).
- 02 La **consommation de tokens** et la **dépendance aux modèles d'IA puissants**.
- 03 L'**humain qui doit rester au cœur des décisions** et contrôler les artefacts .
- 04 La **scalabilité limitée (rigidité)** sur des systems-of-systems.
- 05 Être **pragmatique** : ce n'est **pas un outil magique**.

« La spec est le nouveau source code. **Maîtrisez-la.** »



Merci !

Questions ?

Yohann Ciurlik

Principal Solutions Architect · Capgemini DCX Paris

Giovanni Scamandro

Managing Solutions Architect · Capgemini DCX Lille

Preneurs de vos feedbacks :



SECTION

Annexes

De l'idée au code : et si l'IA était là *à chaque étape* ?

Take away — DCX



LE PROBLÈME

Le vibe coding sans cadre fait perdre le contrôle.

01



Pas de mémoire

L'IA repart de zéro à chaque conversation. Vos décisions d'hier sont perdues.

02



Résultats aléatoires

La même demande donne des résultats différents — impossible de capitaliser.

03



Coût caché

Les erreurs se découvrent tard. Corriger en sprint 3 coûte 10x plus.

04



Perte de contrôle

L'IA part dans une direction, vous dans une autre — personne ne pilote.



LA SOLUTION

Le spec-driven development.
Écrire la spec avant le code.

Une spécification partagée entre vous et l'IA. Fil conducteur, mémoire commune, référence pour tout ce qui suit.

LES 3 NIVEAUX SELON MARTIN FOWLER



NIVEAU 1

Spec-first

La spec guide

Un document écrit avant de coder. L'IA le lit et s'en inspire.



NIVEAU 2

Spec-anchored

La spec pilote

Document maintenu tout au long du projet. L'IA ne code pas sans elle.



NIVEAU 3

Spec-as-source

La spec est le code

BMAD vise le niveau 2 → 3 :
spec vivante, agents spécialisés,
trace complète de chaque décision.



BMAD

Build More Architect Dreams.
Une équipe projet complète,
en agents IA.



CE QUE C'EST

- ✓ Un framework de pilotage de projet avec l'IA
- ✓ Une méthode pour aller de l'idée au code sans perdre le fil
- ✓ Un système qui garde la mémoire de toutes vos décisions
- ✓ Compatible Claude, Cursor, VS Code, Copilot...



CE QUE CE N'EST PAS

- ✗ Un assistant IA classique à qui on pose des questions
- ✗ Un outil réservé aux développeurs seniors
- ✗ Une solution pour remplacer l'équipe
- ✗ Obligatoire pour chaque projet — il y a des alternatives

LES 4 PHASES



01
Discovery
(optionnelle)



02
Planning



03
Solutioning



04
Implementation
(en boucle)



FONCTIONNALITÉS CLÉS

Ce qui rend BMAD vraiment utile au quotidien.



Discovery

Clarifier le besoin avant de construire. Questions itératives, vision produit, personas, critères de succès.



Spécifications vivantes

PRD, architecture, stories : tout est structuré, versionné et toujours à jour.



Agents spécialisés

Analyst, PM, Architect, UX, Developer, Tech Writer... chacun son rôle, coordonné par le BMad Master.



Traçabilité & mémoire

Chaque décision est gardée. Contexte, choix, arbitrages : rien ne se perd.



Qualité par design

Tests, revues de code, standards : intégrés dès le début, pas après.



Pilotage itératif

Boucles courtes, validations régulières, adaptation continue.



EN RÉSUMÉ



De l'idée au code,
sans perdre le fil.



Moins d'aléatoire,
plus de contrôle.



Plus de qualité,
livrée plus vite.

BMAD = la bonne méthode pour tirer
le meilleur de l'IA, à chaque étape.



ANNEXE

Comparaison des approches.

Spec-Kit GitHub « <i>La constitution du projet</i> » Document constitution unique. Process en 4 phases. Intégration Copilot native Simple à démarrer PETITE ÉQUIPE / NOUVEAU PROJET GITHUB	OpenSpec npm · 77K+ « <i>Delta specs minimalistes</i> » Seul ce qui change est spécifié. Compact, pensé brownfield. Ultra léger Brownfield & legacy CODEBASE EXISTANTE / ÉVOLUTIONS FRÉQUENTES	BMAD 43K+ GitHub « <i>L'équipe agile complète</i> » Orchestration multi-agents full SDLC. Personas nommés, audit trail. Cycle complet idée → code Audit trail complet PROJETS COMPLEXES / GREENFIELD ENTERPRISE
---	---	--



ANNEXE

Tableau comparatif détaillé.

CRITÈRE	SPEC-KIT	OPENSPEC	BMAD
Complexité	2 / 5	1 / 5	4 / 5
Multi-agents	Non	Non	Oui
Codebase existante	Partiel	Natif	Partiel
Projet de zéro	Oui	Oui	Excellent
Adversarial Review	Non	Non	Oui
Party Mode	Non	Non	Oui
Trace décisions	Partiel	Partiel	Complète
Prise en main	Facile	Très facile	Progressive
Idéal pour	Petite équipe GitHub	Maintenance / brownfield	Projets complexes